Relatório da Reengenharia

Disciplina: Engenharia de Software – Reengenharia e Engenharia Direta (ciclo de Pressman)

Sistema alvo: classe legada **x_Mgr_F** | LLM utilizado: llama3.1:8b (Ollama, execução local)

Grupo 3

1. Prompt Original

Prompt entregue à IA, já com o dicionário de dados obtido na Engenharia Reversa:

```
Atue como um Arquiteto de Software sênior em Java (Clean Code, SOLID).

Faça a reengenharia da classe legada abaixo, preservando EXATAMENTE o
comportamento:

public class x_Mgr_F {
    public double calc(int tp, double[] vls, boolean st) {
        double r = 0;
        for (int i = 0; i < vls.length; i++) {
            r += vls[i];
        }
        if (tp == 1) {
            r = r - (r * 0.12);
            if (st) {
                r += 15.50;
            }
        } else if (tp == 2) {
            if (r > 500) {
                r = r - (r * 0.05);
            }
        }
        System.out.println("LOG: " + r);
        return r;
    }
}

Dicionário de dados:
- x_Mgr_F -> CalculadoraDeValorFinal
- calc -> calcular
- tp -> tipoConsulta (1 = convênio, 2 = particular)
- vls -> procedimentosRealizados
- st -> aplicaAcrescimoFixo (significado incerto; só controla o acréscimo
  de 15,50 quando tp = 1)
- r -> subtotal / valorFinal

Instruções:
1. Use nomes limpos, sem abreviações.
2. Separe a soma dos procedimentos da aplicação dos descontos (SRP).
3. Use Strategy para convênio e particular, com uma Factory para escolher
   a política.
4. Remova o System.out.println.
5. Use constantes no lugar de números mágicos.
6. Some com laço for (não use stream).

Entregue o código Java completo e uma classe Main testando: tipo 1,
[200.0, 100.0], acréscimo verdadeiro (esperado 279.5).
```

2. Análise Crítica e Ajustes Manuais

A primeira resposta do LLM foi analisada pelo grupo antes de ser aceita. Ela não preservava o comportamento do legado. Os problemas encontrados e as correções feitas foram:

Problema na saída do LLM	Ajuste manual

Não compilava: `double[]` não possui `.stream()`.	Troca por laço `for` simples (também evita diferenças numéricas do `DoubleStream.sum()`).
Acréscimo de 15,50 tratado como desconto (sinal invertido): o exemplo da atividade resultava em 248,50 em vez de 279,50.	Acréscimo somado ao valor **depois** do desconto, como no legado.
O parâmetro `st` era ignorado.	Renomeado para `aplicaAcrescimoFixo` e usado somente no convênio.
Qualquer `tp` diferente de 1 era tratado como particular.	Criada a `CobrancaSemAjuste` para os demais tipos, como no legado.
Nomes ruins (`ConvencionalParticularDesconto`) e abreviações mantidas (tp, vls, st).	Nomes baseados no dicionário de dados: tipoConsulta, procedimentosRealizados etc.
Uso de if/else espalhado, sem extensibilidade.	Aplicados Strategy (uma política por tipo) e Factory para escolher a política.
`System.out.println` misturado à regra de negócio.	Removido da regra. O log não faz parte do cálculo.

Observação: o significado de `st` permanece incerto no dicionário de dados do grupo. Por isso foi usado um nome neutro, descrevendo apenas o efeito comprovado no código, sem inventar regra de negócio.

3. Validação de Manutenção de Comportamento

A classe `Main` executa o código novo e o legado com as mesmas entradas e compara os resultados com o valor calculado à mão. Os 5 casos cobrem todos os ramos do legado:

Caso	Entrada	Novo	Legado	Esperado
Convênio com acréscimo	tp 1, [200, 100], st true	279.5	279.5	279.5
Convênio sem acréscimo	tp 1, [200, 100], st false	264.0	264.0	264.0
Particular acima de 500	tp 2, [600]	570.0	570.0	570.0
Particular até 500	tp 2, [400]	400.0	400.0	400.0
Outro tipo	tp 3, [100]	100.0	100.0	100.0

Saída da execução (todos os casos retornaram OK):

```
Convênio com acréscimo -> novo=279.5 | legado=279.5 | esperado=279.5 | OK
Convênio sem acréscimo -> novo=264.0 | legado=264.0 | esperado=264.0 | OK
Particular acima de 500 -> novo=570.0 | legado=570.0 | esperado=570.0 | OK
Particular até 500 -> novo=400.0 | legado=400.0 | esperado=400.0 | OK
Outro tipo (sem regra) -> novo=100.0 | legado=100.0 | esperado=100.0 | OK
```

4. Comparativo Arquitetural

A classe legada `x_Mgr_F` concentra em um único método a soma dos valores, as regras de desconto de convênio e particular, o acréscimo fixo e a escrita de log, além de usar nomes sem

significado (`tp`, `vls`, `st`, `r`) e números mágicos (0.12, 0.05, 500, 15.50). Isso é o que Fowler descreve como *Long Method* e *Magic Number*, e viola o **Princípio de Responsabilidade Única (SRP)**: o método tem vários motivos para mudar.

A estrutura proposta é superior por quatro razões. **(1) SRP:** a soma fica em `CalculadoraDeValorFinal` e cada regra de desconto fica em sua própria classe, de modo que mudar o desconto do convênio não afeta o particular. **(2) Aberto/Fechado (OCP):** um novo tipo de convênio exige apenas uma nova classe que implemente `PoliticaDeCobranca` e um registro na `PoliticaDeCobrancaFactory`, sem alterar as políticas existentes. **(3) Legibilidade e refatorações de Fowler:** foram aplicados *Extract Method/Class*, *Rename*, *Replace Magic Number with Symbolic Constant* e *Replace Conditional with Polymorphism* (Strategy), eliminando a cadeia de if/else. **(4) Testabilidade:** sem o `System.out.println` na regra, cada política pode ser testada isoladamente, sem efeitos colaterais. Como a validação dinâmica mostrou resultados idênticos aos do legado nos 5 cenários, a reengenharia modernizou a estrutura sem alterar o comportamento funcional, o que reduz o custo de manutenção a longo prazo.

5. Código Final

PoliticaDeCobranca.java

```
/**
 * Contrato (Strategy) de uma política de cobrança.
 * Cada tipo de consulta implementa a sua própria regra.
 */
public interface PoliticaDeCobranca {

    double calcular(double subtotal, boolean aplicaAcrescimoFixo);
}
```

CobrancaConvenio.java

```
/**
 * Política do tipo de consulta 1 (convênio).
 * Aplica 12% de desconto e, depois do desconto, o acréscimo fixo (se solicitado).
 */
public class CobrancaConvenio implements PoliticaDeCobranca {

    private static final double PERCENTUAL_DESCONTO = 0.12;
    private static final double ACRESCIMO_FIXO = 15.50;

    @Override
    public double calcular(double subtotal, boolean aplicaAcrescimoFixo) {
        double valorComDesconto = subtotal - (subtotal * PERCENTUAL_DESCONTO);
        if (aplicaAcrescimoFixo) {
            return valorComDesconto + ACRESCIMO_FIXO;
        }
        return valorComDesconto;
    }
}
```

CobrancaParticular.java

```
/**
 * Política do tipo de consulta 2 (particular).
 * Aplica 5% de desconto somente quando o subtotal ultrapassa o limite.
 */
public class CobrancaParticular implements PoliticaDeCobranca {

    private static final double LIMITE_PARA_DESCONTO = 500.0;
    private static final double PERCENTUAL_DESCONTO = 0.05;

    @Override
```

```
    public double calcular(double subtotal, boolean aplicaAcrescimoFixo) {
        if (subtotal > LIMITE_PARA_DESCONTO) {
            return subtotal - (subtotal * PERCENTUAL_DESCONTO);
        }
        return subtotal;
    }
}
```

CobrancaSemAjuste.java

```
/**
 * Política para qualquer outro tipo de consulta: o legado não define regra,
 * então o valor permanece igual ao subtotal.
 */
public class CobrancaSemAjuste implements PoliticaDeCobranca {

    @Override
    public double calcular(double subtotal, boolean aplicaAcrescimoFixo) {
        return subtotal;
    }
}
```

PoliticaDeCobrancaFactory.java

```
/**
 * Factory responsável por escolher a política de cobrança conforme o tipo de consulta.
 * Para criar um novo tipo, basta criar uma nova política e registrá-la aqui.
 */
public class PoliticaDeCobrancaFactory {

    public static final int TIPO_CONVENIO = 1;
    public static final int TIPO_PARTICULAR = 2;

    public static PoliticaDeCobranca criar(int tipoConsulta) {
        switch (tipoConsulta) {
            case TIPO_CONVENIO:
                return new CobrancaConvenio();
            case TIPO_PARTICULAR:
                return new CobrancaParticular();
            default:
                return new CobrancaSemAjuste();
        }
    }
}
```

CalculadoraDeValorFinal.java

```
/**
 * Orquestra o cálculo: soma os procedimentos e delega o ajuste
 * à política de cobrança adequada.
 * (Substitui a classe legada x_Mgr_F, sem efeitos colaterais como System.out.println.)
 */
public class CalculadoraDeValorFinal {

    public double calcular(int tipoConsulta,
                           double[] procedimentosRealizados,
                           boolean aplicaAcrescimoFixo) {
        double subtotal = somarProcedimentos(procedimentosRealizados);
        PoliticaDeCobranca politica = PoliticaDeCobrancaFactory.criar(tipoConsulta);
        return politica.calcular(subtotal, aplicaAcrescimoFixo);
    }

    private double somarProcedimentos(double[] procedimentosRealizados) {
        double soma = 0;
        for (int i = 0; i < procedimentosRealizados.length; i++) {
            soma += procedimentosRealizados[i];
        }
        return soma;
    }
}
```

Main.java

```
/**
 * Validação de manutenção de comportamento: compara o código novo
 * com o legado e com o valor esperado calculado à mão.
 */
public class Main {

    public static void main(String[] args) {
        CalculadoraDeValorFinal novo = new CalculadoraDeValorFinal();
        x_Mgr_F legado = new x_Mgr_F();

        // tipoConsulta, procedimentos, aplicaAcrescimoFixo, esperado
        testar(novo, legado, "Convênio com acréscimo", 1, new double[]{200.0, 100.0}, true, 279.5);
        testar(novo, legado, "Convênio sem acréscimo", 1, new double[]{200.0, 100.0}, false, 264.0);
        testar(novo, legado, "Particular acima de 500", 2, new double[]{600.0}, false, 570.0);
        testar(novo, legado, "Particular até 500", 2, new double[]{400.0}, false, 400.0);
        testar(novo, legado, "Outro tipo (sem regra)", 3, new double[]{100.0}, true, 100.0);
    }

    private static void testar(CalculadoraDeValorFinal novo, x_Mgr_F legado, String nome,
                               int tipo, double[] valores, boolean acrescimo, double esperado) {
        double resultadoNovo = novo.calcular(tipo, valores, acrescimo);
        double resultadoLegado = legado.calc(tipo, valores, acrescimo);
        boolean ok = resultadoNovo == esperado && resultadoNovo == resultadoLegado;
        System.out.println(nome + " -> novo=" + resultadoNovo
                + " | legado=" + resultadoLegado
                + " | esperado=" + esperado
                + " | " + (ok ? "OK" : "FALHOU"));
    }
}
```